

Image Analysis — Assignment 1

(Solutions and Report)

Student:

Programme: MLSC

September 2025

Contents

1	Image sampling	2
2	Histogram equalization	2
3	Neighbourhood of pixels	3
4	OCR1: Segmentation	4
5	Dimensionality	15
6	Scalar products and norm on images	16
7	Image Compression (PCA Basis Projection)	17
8	Image bases	19

1 Image sampling

Continuous image: $f(x, y) = x(1 - y)$ for $0 \leq x, y \leq 1$. Evenly sample to 7×7 pixels with the lower-left pixel at $(0, 0)$ and upper-right at $(1, 1)$. Quantize to 16 gray levels $\{0, \dots, 15\}$ by $q = \text{round}(15f)$.

The sampled continuous matrix F (rows bottom→top) has entries $F_{j,i} = x_i(1 - y_j)$ with $x_i = y_i = \frac{i}{6}$, $i = 0, \dots, 6$. The quantized 7×7 image $Q = \text{round}(15F)$ is

$$Q = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 & 2 & 2 & 3 \\ 0 & 1 & 2 & 3 & 3 & 4 & 5 \\ 0 & 1 & 2 & 4 & 5 & 6 & 8 \\ 0 & 2 & 3 & 5 & 7 & 8 & 10 \\ 0 & 2 & 4 & 6 & 8 & 10 & 12 \\ 0 & 2 & 5 & 8 & 10 & 12 & 15 \end{bmatrix}.$$

Explanation. Uniform sampling gives a bilinear ramp decreasing with y and increasing with x ; uniform 4-bit quantization maps $0 \mapsto 0$, $1 \mapsto 15$.

2 Histogram equalization

The input gray-level probability density function is

$$p_r(r) = 6r(1 - r), \quad r \in [0, 1].$$

In histogram equalization, the cumulative distribution functions before and after the transformation must satisfy

$$\int_0^r p_r(w) dw = \int_0^s p_s(u) du,$$

where $s = T(r)$, and the desired output distribution is uniform:

$$p_s(s) = 1, \quad s \in [0, 1].$$

Thus,

$$\int_0^r 6w(1 - w) dw = \int_0^s 1 du.$$

Compute the left-hand side:

$$\int_0^r 6w(1 - w) dw = \int_0^r (6w - 6w^2) dw = \left[3w^2 - 2w^3 \right]_0^r = 3r^2 - 2r^3.$$

The right-hand side is

$$\int_0^s 1 du = s.$$

Therefore the transformation is

$$s = T(r) = 3r^2 - 2r^3, \quad r \in [0, 1].$$

$s = 3r^2 - 2r^3$

3 Neighbourhood of pixels

Threshold $g(i, j) = \mathbf{1}[f(i, j) > 1]$ for the given 15×15 image f .

(a) Thresholded image g .

$$g = \begin{bmatrix} 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 1 \\ 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 \\ 1 & 0 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 & 1 & 1 & 0 & 0 & 0 & 1 & 1 & 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 0 & 1 \\ 1 & 0 & 0 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 0 & 0 & 1 \\ 1 & 1 & 0 & 0 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 0 & 0 & 1 & 1 \\ 1 & 1 & 1 & 0 & 0 & 1 & 1 & 1 & 1 & 1 & 0 & 0 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \end{bmatrix}$$

(b) **4-connected components (for $g = 1$).** After applying 4-neighborhood connectivity, we obtain in total **4** connected components. The labeled image (components marked as A–D) is:

$$\begin{bmatrix} A & A & A & A & A & A & A & A & A & A & A & A & A & A & A \\ A & A & 0 & A & 0 & 0 & 0 & 0 & 0 & 0 & 0 & A & A & A & A \\ A & A & A & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & A & A & A \\ A & A & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & A & A \\ A & 0 & 0 & B & B & 0 & 0 & 0 & 0 & C & C & 0 & 0 & 0 & A \\ A & 0 & 0 & B & B & B & 0 & 0 & 0 & C & C & C & 0 & 0 & A \\ A & 0 & 0 & B & B & 0 & 0 & 0 & 0 & C & C & 0 & 0 & 0 & A \\ A & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & A \\ A & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & A \\ A & 0 & 0 & D & 0 & 0 & 0 & 0 & 0 & 0 & 0 & D & D & 0 & A \\ A & 0 & 0 & D & D & D & D & D & D & D & D & D & 0 & 0 & A \\ A & A & 0 & 0 & D & D & D & D & D & D & D & 0 & 0 & A & A \\ A & A & A & 0 & 0 & D & D & D & D & D & 0 & 0 & A & A & A \\ A & A & A & A & 0 & 0 & 0 & 0 & 0 & 0 & 0 & A & A & A & A \\ A & A & A & A & A & A & A & A & A & A & A & A & A & A & A \end{bmatrix}$$

4 OCR1: Segmentation

(Python)

Segmentation method. Given a grayscale image I (light digits on dark background), `im2segment` returns a list of binary masks $[S_1, \dots, S_n]$, each $S_k \in \{0, 1\}^{H \times W}$ with the same size as I , one per digit. The pipeline is as follows in a single pass: we first convert to grayscale if needed, normalize to $[0, 1]$, and apply global Otsu thresholding to obtain a coarse foreground F . To bridge short horizontal gaps we perform a light morphological *closing* with a 1×3 structuring element, yielding F' . We then compute a column projection on F' and mark a column as active if its foreground count $\geq \max(2, 0.02H)$; consecutive active columns form runs, and neighboring runs separated by a gap $\leq \text{GAP_MAX} = 0.33H$ (measured in columns) are merged. For each run we crop the block and perform 8-connected component labeling; letting $A_{\max}$ be the largest component area and $\alpha = 0.20$, we keep the union of the largest component and any other component with area $\geq \alpha A_{\max}$, discarding smaller fragments as noise, and write this union back into a full-size mask. The resulting masks are naturally ordered left-to-right by the runs.

Listing 1: `im2segment.py`

```
1 import numpy as np
2 import scipy.ndimage as ndi
3 import matplotlib.pyplot as plt
4
5 def im2segment(I, show_steps=False, connectivity=8):
6     """
7     Segment a light-on-dark text image into a list of instance masks (one
8     digit per mask).
9
10    Returns a list where each element is a binary (0/1) image with the same
11    size as the input.
12
13    Parameters
14    -----
15    I : np.ndarray
16        Input image. Either HxW grayscale or HxWx3 color.
17    show_steps : bool
18        If True, visualize key intermediate results (Otsu binary, closing)
19        and each output mask.
20    connectivity : int
21        Connectivity option. The current implementation uses 8-connectivity
22        (explicit 8-neighborhood).
23
24    """
25    # ----- Preprocess: to grayscale + normalize to [0,1] -----
26    A = I.astype(np.float32)
27    if A.ndim == 3:                # convert color to grayscale
```

```

22     A = A.mean(axis=2)
23     if A.max() > 1.5:
24         A /= 255.0
25
26     # ----- Otsu thresholding -----
27     hist, _ = np.histogram(A.ravel(), bins=256, range=(0, 1))
28     total = A.size
29     sumB = wB = maximum = 0.0
30     sum1 = np.dot(np.arange(256), hist)
31     level = 128
32     for t in range(256):
33         wB += hist[t]
34         if wB == 0:
35             continue
36         wF = total - wB
37         if wF == 0:
38             break
39         sumB += t * hist[t]
40         mB = sumB / wB
41         mF = (sum1 - sumB) / wF
42         between = wB * wF * (mB - mF) ** 2
43         if between >= maximum:
44             level, maximum = t, between
45     fg_otsu = (A >= level / 255.0)
46
47     # ----- Morphological closing -----
48     fg_close = ndi.binary_closing(fg_otsu, structure=np.ones((1, 3)))
49     fg = fg_close
50
51     # ----- Column projection to find active column runs -----
52     H, W = fg.shape
53     colsum = fg.sum(axis=0)
54     active = colsum >= max(2, int(0.02 * H)) # adaptive threshold
55
56     runs, start = [], None
57     for x, a in enumerate(active):
58         if a and start is None:
59             start = x
60         if (not a) and start is not None:
61             runs.append((start, x - 1))
62             start = None
63     if start is not None:
64         runs.append((start, W - 1))
65
66     # Merge small gaps between neighboring runs

```

```

67     GAP_MAX = int(0.33 * H)    # tunable (H = image height, GAP_MAX measured
                                # in columns)
68     merged = []
69     for r in runs:
70         if not merged or r[0] - merged[-1][1] > GAP_MAX:
71             merged.append(list(r))
72         else:
73             merged[-1][1] = r[1]
74
75     # ----- Within each run, keep the largest connected components
76     # -----
77     S = []
78     structure = np.ones((3, 3), dtype=bool) if connectivity == 8 else None
79     for x0, x1 in merged:
80         x1, xr = max(0, x0 - 1), min(W - 1, x1 + 1)
81         block = fg[:, x1:xr + 1]
82         labels, nlab = ndi.label(block, structure=structure)
83         if nlab == 0:
84             continue
85
86         # Area filtering: keep the union of "largest component +
87         # components with area alpha * max_area"
88         areas = np.array([(labels == k).sum() for k in range(1, nlab + 1)])
89         max_area = areas.max()
90         alpha = 0.20 # area relaxation ratio
91         keep_ids = [i + 1 for i, a in enumerate(areas) if a >= alpha *
92                     max_area]
93
94         merged_mask = np.isin(labels, keep_ids)
95
96         mask = np.zeros_like(fg, dtype=np.uint8)
97         mask[:, x1:xr + 1] = merged_mask.astype(np.uint8)
98         S.append(mask)
99
100     # ----- Visualization -----
101     if show_steps:
102         n_masks = len(S)
103         plt.figure(figsize=(3 * (3 + n_masks), 3))
104
105         plt.subplot(1, 3 + n_masks, 1); plt.imshow(A, cmap='gray');
106         plt.title("Input"); plt.axis('off')
107         plt.subplot(1, 3 + n_masks, 2); plt.imshow(fg_otsu, cmap='gray');
108         plt.title("After Otsu"); plt.axis('off')
109         plt.subplot(1, 3 + n_masks, 3); plt.imshow(fg_close, cmap='gray');
110         plt.title("After Closing"); plt.axis('off')

```

```

105
106     for i, m in enumerate(S):
107         plt.subplot(1, 3 + n_masks, 4 + i)
108         plt.imshow(m, cmap='gray'); plt.title(f"Mask {i}"); plt.axis('
            off')
109
110     plt.tight_layout(); plt.show()
111
112     return S

```

Baseline Results

(1) Text Output Results

```

1 All Jaccard indices:
2 (np.float64(0.6540880503144654), np.float64(0.5961538461538461), np.float64
    (0.6867469879518072), np.float64(0.36134453781512604), np.float64
    (0.8161764705882353), np.float64(0.6363636363636364), np.float64
    (0.7575757575757576), np.float64(0.5984251968503937), np.float64
    (0.7375886524822695), np.float64(0.7022900763358778), np.float64
    (0.6951219512195121), np.float64(0.7159090909090909), np.float64
    (0.7073170731707317), np.float64(0.7153846153846154), np.float64
    (0.559322033898305), np.float64(0.28431372549019607), np.float64
    (0.6014492753623188), np.float64(0.6629213483146067), np.float64
    (0.8090909090909091), np.float64(0.7482993197278912), np.float64
    (0.7973568281938326), np.float64(0.8153153153153153), np.float64
    (0.6666666666666666), np.float64(0.5157894736842106), np.float64
    (0.6573426573426573), np.float64(0.7557251908396947), np.float64
    (0.6909090909090909), np.float64(0.6510067114093959), np.float64
    (0.7116564417177914), np.float64(0.7241379310344828), np.float64
    (0.7731958762886598), np.float64(0.5903614457831325), np.float64
    (0.6785714285714286), np.float64(0.6544117647058824), np.float64
    (0.6810344827586207), np.float64(0.7888198757763976), np.float64
    (0.7678571428571429), np.float64(0.26), np.float64(0.8087431693989071),
    np.float64(0.5255474452554745), np.float64(0.5737704918032787), np.
    float64(0.689922480620155), np.float64(0.7559523809523809), np.float64
    (0.8083333333333333), np.float64(0.6506024096385542), np.float64
    (0.7570093457943925), np.float64(0.2631578947368421), np.float64
    (0.5795454545454546), np.float64(0.7961165048543689), np.float64(0.38))
3 All total Jaccard indices (total for each image):
4 (np.float64(0.17875770507349456), np.float64(0.18292682926829268), np.
    float64(0.1838474025974026), np.float64(0.178743961352657), np.float64
    (0.1851389322510147), np.float64(0.18383838383838383), np.float64
    (0.18328506412908566), np.float64(0.18103137396321672), np.float64
    (0.18407750631844988), np.float64(0.1719512195121951))

```


(2) Image Output Results

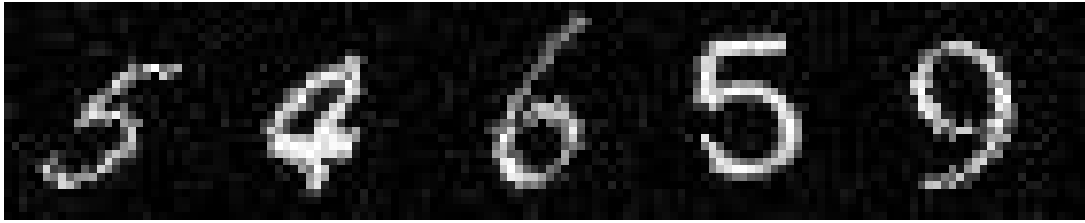


Figure 1: Input Image

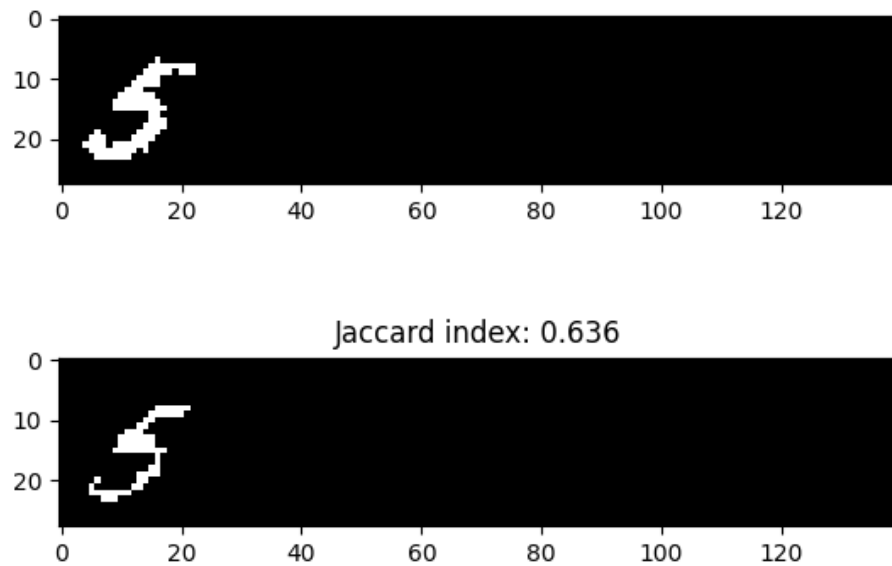


Figure 2: Segmentation Result S_1

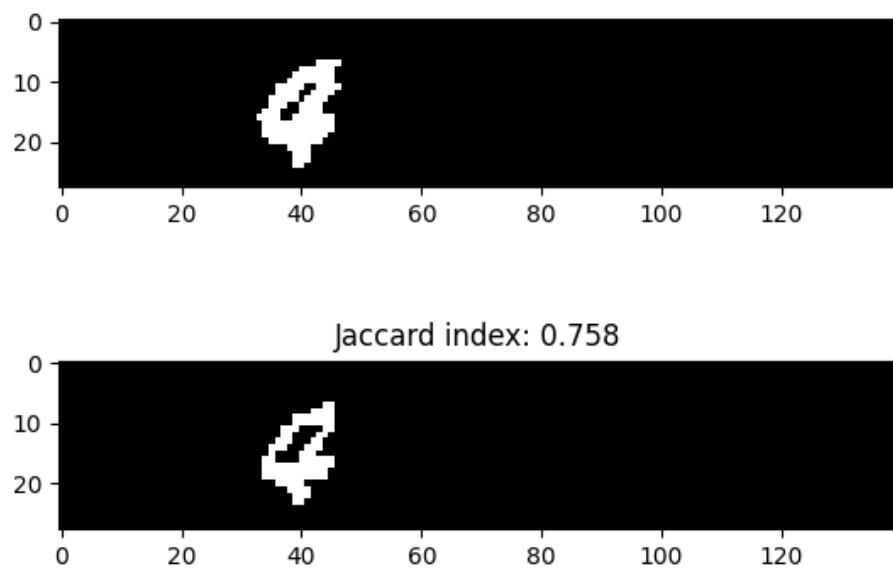


Figure 3: Segmentation Result S_2

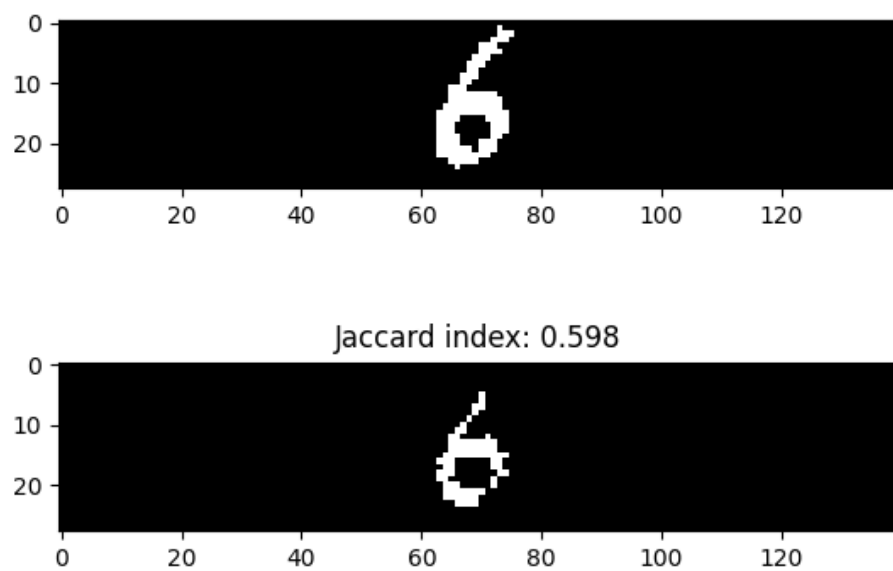


Figure 4: Segmentation Result S_3

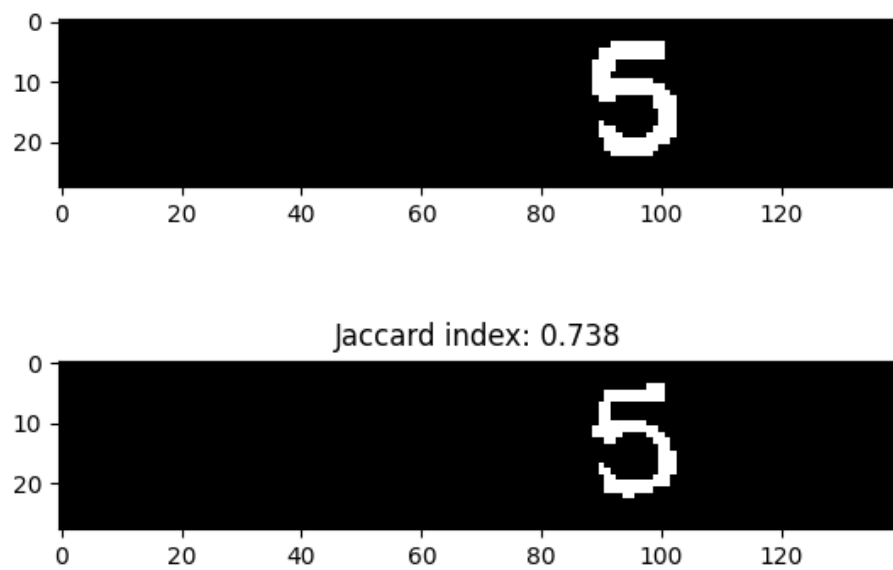


Figure 5: Segmentation Result S_4

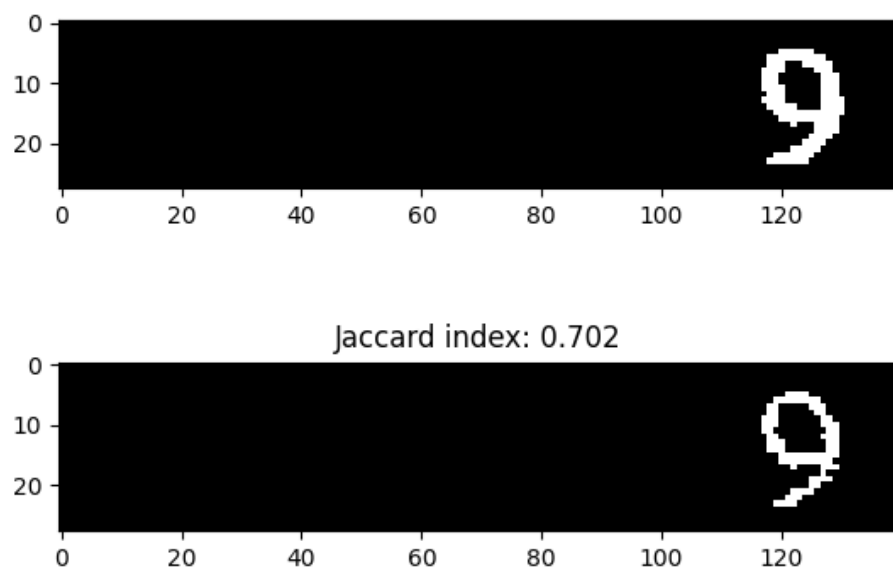


Figure 6: Segmentation Result S_5

Motivation for a learning-based model. The global Otsu baseline is simple and fast, but it tends to degrade under uneven illumination, touching digits, or thin strokes. To improve robustness, I additionally trained a U-Net (ResNet-18 encoder, 1-channel input, 11 classes including background) on the M2NIST dataset and then used the trained weights to segment the same benchmark images. The post-processing converts the semantic map into instance masks (one mask per digit).

(3) U-Net Text Output Results

```

1 All Jaccard indices:
2 (np.float64(0.8895348837209303), np.float64(0.7913043478260869), np.float64
  (0.77), np.float64(0.78125), np.float64(0.9178082191780822), np.float64
  (0.8348623853211009), np.float64(0.9473684210526315), np.float64
  (0.8805970149253731), np.float64(0.9395973154362416), np.float64
  (0.8897058823529411), np.float64(0.8932584269662921), np.float64
  (0.8617021276595744), np.float64(0.8636363636363636), np.float64
  (0.9624060150375939), np.float64(0.873015873015873), np.float64
  (0.6993006993006993), np.float64(0.8827586206896552), np.float64
  (0.8602150537634409), np.float64(0.8717948717948718), np.float64
  (0.9466666666666667), np.float64(0.9439655172413793), np.float64
  (0.8947368421052632), np.float64(0.9136690647482014), np.float64
  (0.8108108108108109), np.float64(0.8783783783783784), np.float64
  (0.9323308270676691), np.float64(0.8245614035087719), np.float64
  (0.9285714285714286), np.float64(0.9034090909090909), np.float64
  (0.9054054054054054), np.float64(0.8894230769230769), np.float64
  (0.8061224489795918), np.float64(0.8916666666666667), np.float64
  (0.9251700680272109), np.float64(0.8387096774193549), np.float64
  (0.8888888888888888), np.float64(0.8869047619047619), np.float64
  (0.7222222222222222), np.float64(0.8974358974358975), np.float64
  (0.8978102189781022), np.float64(0.8837209302325582), np.float64
  (0.851063829787234), np.float64(0.9085714285714286), np.float64
  (0.9147286821705426), np.float64(0.8850574712643678), np.float64
  (0.8983050847457628), np.float64(0.24786324786324787), np.float64(0.0),
  np.float64(0.0), np.float64(0.0))
3 All total Jaccard indices (total for each image):
4 (np.float64(0.17782761935277155), np.float64(0.18759836323575702), np.
  float64(0.1853629274145171), np.float64(0.1776061776061776), np.float64
  (0.18753044325377496), np.float64(0.189214463840399), np.float64
  (0.1813594538438706), np.float64(0.18686171680927408), np.float64
  (0.1853135833595966), np.float64(0.163727959697733))
5 Total mean Jaccard score is 0.81

```

(4) U-Net Image Output Results

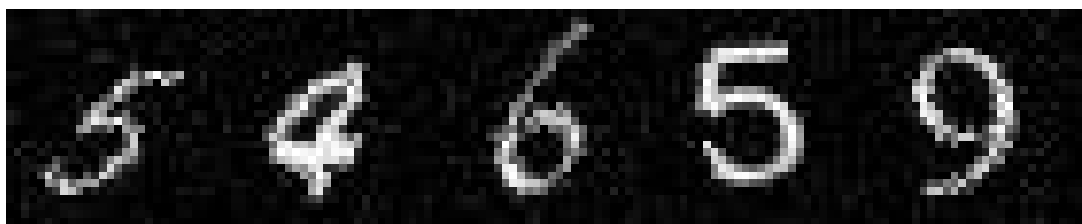


Figure 7: Input Image

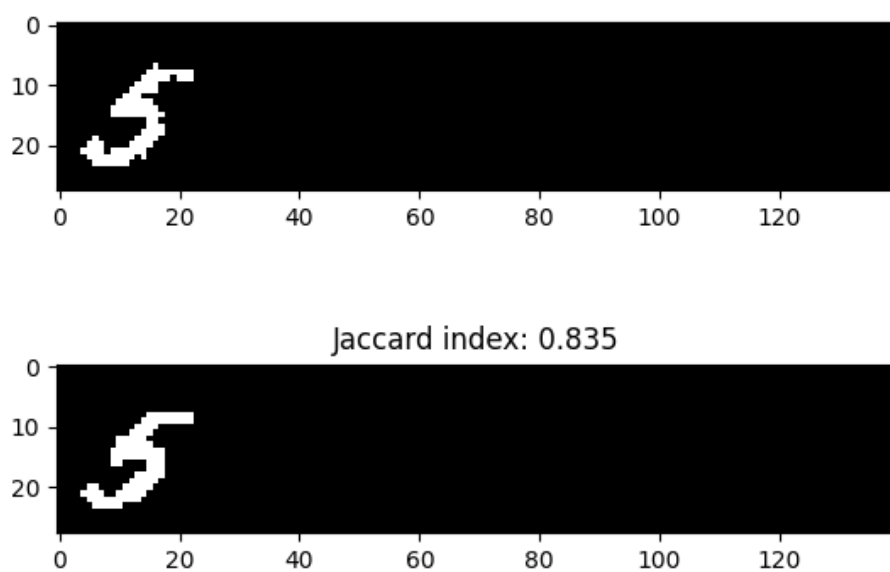


Figure 8: Segmentation Result S_1

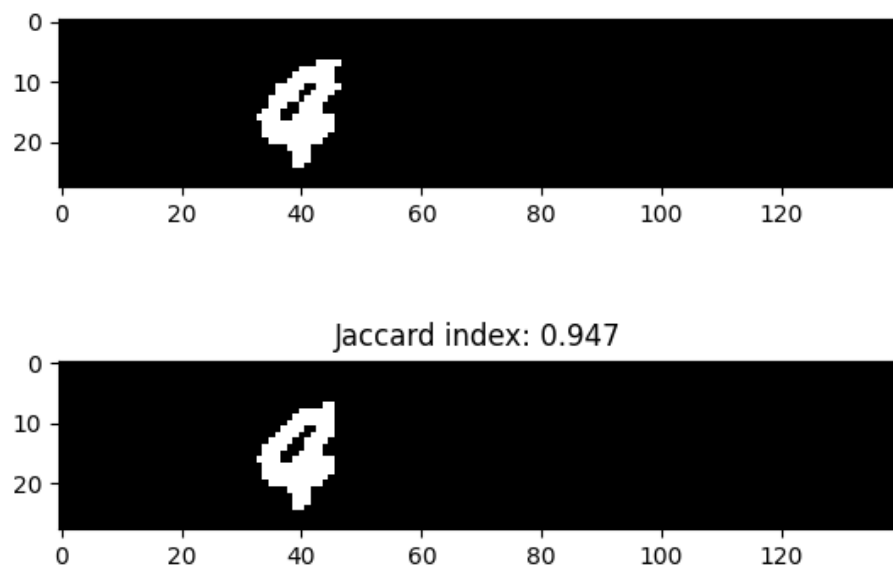


Figure 9: Segmentation Result S_2

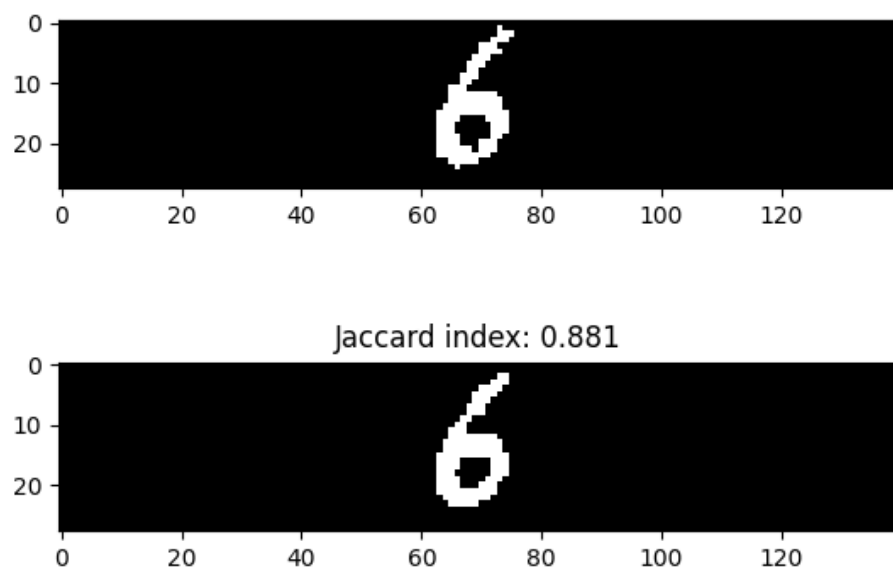


Figure 10: Segmentation Result S_3

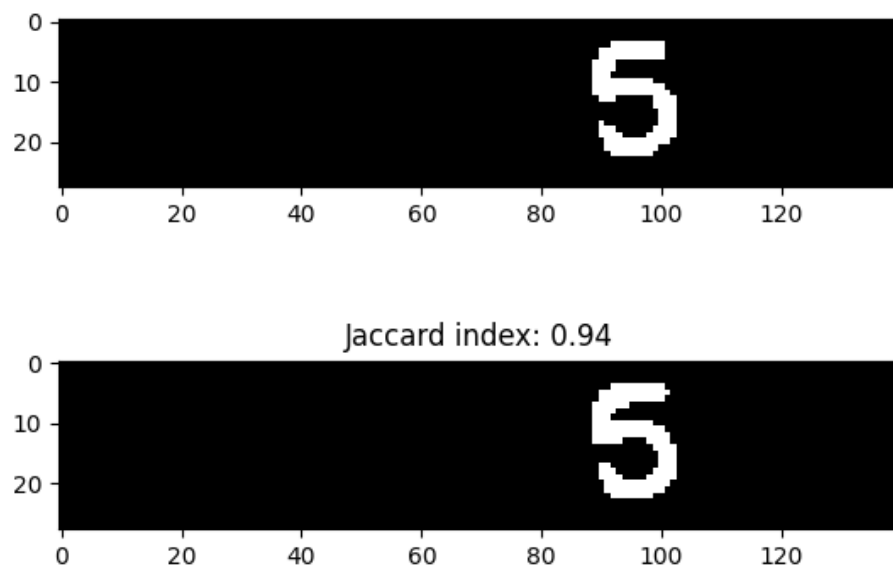


Figure 11: Segmentation Result S_4

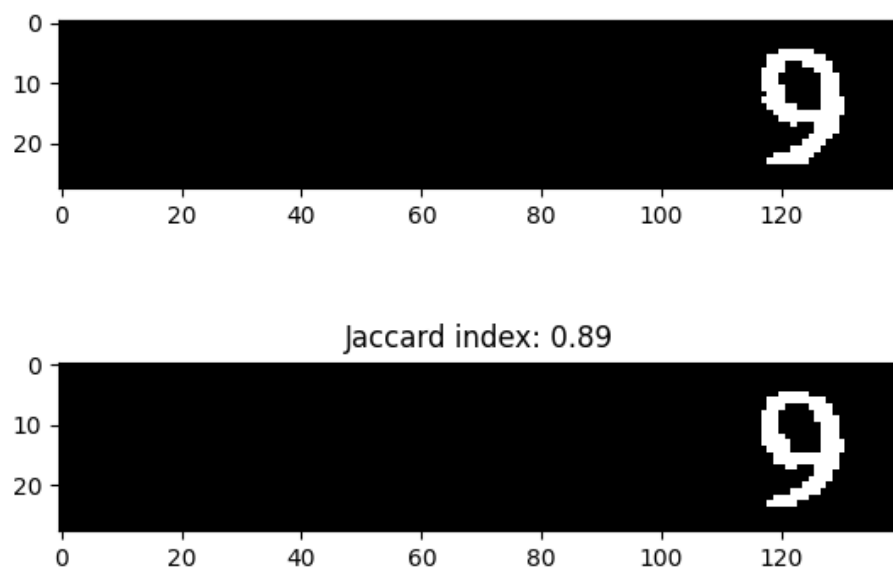


Figure 12: Segmentation Result S_5

5 Dimensionality

A. 2×4 grayscale image space

Dimension $k : 2 \times 4 = 8$.

One possible basis (explicitly writing $e_1, \dots, e_8$): take the standard basis, where each basis vector is a 2×4 image with a single pixel equal to 1 and all others equal to 0 (over the scalar field $\mathbb{R}$). For example, using row-major ordering:

$$e_1 = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}, \quad e_2 = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}, \quad e_3 = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}, \quad e_4 = \begin{bmatrix} 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix},$$

$$e_5 = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix}, \quad e_6 = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}, \quad e_7 = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix}, \quad e_8 = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

Any image u can be uniquely represented as

$$u = \sum_{i=1}^8 x_i e_i,$$

where x_i is the grayscale value of the corresponding pixel.

B. 1200×3000 grayscale image space

Dimension $k : 1200 \times 3000 = 3,600,000$.

Choice and construction of the basis: again, take the standard basis. Pixels are indexed by two-dimensional coordinates (r, c) (row, column), or equivalently flattened into a one-dimensional index using row-major ordering:

$$i = (r - 1) \times 3000 + c.$$

The first few basis elements:

e_1 : value 1 at pixel $(1, 1)$, and 0 elsewhere,

e_2 : value 1 at pixel $(1, 2)$, and 0 elsewhere,

e_3 : value 1 at pixel $(1, 3)$, and 0 elsewhere, and so on.

General construction: for any (r, c) , define

$$e_{(r,c)}(i, j) = \begin{cases} 1, & (i, j) = (r, c), \\ 0, & \text{otherwise.} \end{cases}$$

All 1200×3000 such “unit impulse images” form a basis.

6 Scalar products and norm on images

Scalar product (images as matrices). For images (matrices) $A, B \in \mathbb{R}^{m \times n}$, the scalar product (Frobenius inner product) is defined as

$$\langle A, B \rangle = \sum_{i=1}^m \sum_{j=1}^n A_{ij} B_{ij}.$$

Norm (Frobenius norm). The norm of an image A is

$$\|A\| = \sqrt{\langle A, A \rangle} = \sqrt{\sum_{i,j} A_{ij}^2}.$$

Given

$$u = \begin{bmatrix} 4 & -3 \\ 2 & -1 \end{bmatrix}, \quad v = \frac{1}{2} \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix}, \quad w = \frac{1}{2} \begin{bmatrix} 1 & -1 \\ 1 & -1 \end{bmatrix}.$$

Computations

$$\begin{aligned} \|u\| &= \sqrt{4^2 + (-3)^2 + 2^2 + (-1)^2} = \sqrt{30} \approx 5.4772, \\ \|v\| &= \sqrt{\frac{1}{4}(1^2 + (-1)^2 + (-1)^2 + 1^2)} = 1, \quad \|w\| = \sqrt{\frac{1}{4}(1^2 + (-1)^2 + 1^2 + (-1)^2)} = 1, \\ u \cdot v &= \langle u, v \rangle = \frac{1}{2}(4 \cdot 1 + (-3) \cdot (-1) + 2 \cdot (-1) + (-1) \cdot 1) = \frac{1}{2}(4 + 3 - 2 - 1) = 2, \\ u \cdot w &= \langle u, w \rangle = \frac{1}{2}(4 \cdot 1 + (-3) \cdot (-1) + 2 \cdot 1 + (-1) \cdot (-1)) = \frac{1}{2}(4 + 3 + 2 + 1) = 5, \\ v \cdot w &= \langle v, w \rangle = \frac{1}{4}(1 + 1 - 1 - 1) = 0. \end{aligned}$$

Orthonormality of $\{v, w\}$

Since $\|v\| = \|w\| = 1$ and $\langle v, w \rangle = 0$, the set $\{v, w\}$ is **orthonormal**.

Projection of u onto $\text{span}\{v, w\}$

Because $\{v, w\}$ is orthonormal, the projection is

$$P_{\{v, w\}}(u) = (u \cdot v) v + (u \cdot w) w = 2v + 5w = \begin{bmatrix} 3.5 & -3.5 \\ 1.5 & -1.5 \end{bmatrix}.$$

Approximation quality. The residual is

$$u - P_{\{v, w\}}(u) = \begin{bmatrix} 0.5 & 0.5 \\ 0.5 & 0.5 \end{bmatrix}, \quad \|u - P_{\{v, w\}}(u)\| = \sqrt{4 \times 0.5^2} = 1.$$

Relative error:

$$\frac{\|u - P_{\{v, w\}}(u)\|}{\|u\|} = \frac{1}{\sqrt{30}} \approx 0.1826 \text{ } (\approx 18.3\%).$$

Thus, the projection provides a **reasonable** approximation: the error is about 18% of $\|u\|$.

7 Image Compression (PCA Basis Projection)

Let the four basis images be

$$\varphi_1 = \frac{1}{3} \begin{pmatrix} 1 & 1 & 1 \\ 1 & 0 & 1 \\ -1 & -1 & -1 \\ 0 & -1 & 0 \end{pmatrix}, \quad \varphi_2 = \frac{1}{2} \begin{pmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix}, \quad \varphi_3 = \frac{1}{2} \begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{pmatrix}, \quad \varphi_4 = \frac{1}{3} \begin{pmatrix} 0 & 1 & 0 \\ 1 & 1 & 1 \\ 1 & 0 & 1 \\ 1 & 1 & 1 \end{pmatrix},$$

and the original image

$$f = \begin{pmatrix} 11 & 5 & -3 \\ 12 & 1 & -1 \\ 1 & -4 & -6 \\ 5 & -3 & -2 \end{pmatrix} \in \mathbb{R}^{4 \times 3}.$$

Definition of inner product and norm For matrices $A, B \in \mathbb{R}^{4 \times 3}$, we use the Frobenius inner product

$$\langle A, B \rangle = \sum_{i=1}^4 \sum_{j=1}^3 A_{ij} B_{ij}, \quad \|A\| = \sqrt{\langle A, A \rangle}.$$

(1) Proof that $\{\varphi_1, \varphi_2, \varphi_3, \varphi_4\}$ are orthonormal First, the norms are

$$\|\varphi_1\|^2 = \frac{1}{9} \sum_{ij} \begin{pmatrix} 1 & 1 & 1 \\ 1 & 0 & 1 \\ -1 & -1 & -1 \\ 0 & -1 & 0 \end{pmatrix}_{ij}^2 = \frac{1}{9} \cdot 9 = 1, \quad \|\varphi_2\|^2 = \frac{1}{4} \cdot 4 = 1, \quad \|\varphi_3\|^2 = \frac{1}{4} \cdot 4 = 1, \quad \|\varphi_4\|^2 = \frac{1}{9} \cdot 9 = 1.$$

Then write out *all* cross inner products:

$$\langle \varphi_1, \varphi_2 \rangle = \frac{1}{3} \frac{1}{2} \sum_{i,j} \begin{pmatrix} 1 & 1 & 1 \\ 1 & 0 & 1 \\ -1 & -1 & -1 \\ 0 & -1 & 0 \end{pmatrix}_{ij} \begin{pmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix}_{ij} = \frac{1}{6} [(1-1) + (1-1) + 0 + 0] = 0.$$

$$\langle \varphi_1, \varphi_3 \rangle = \frac{1}{3} \frac{1}{2} \sum_{i,j} \begin{pmatrix} 1 & 1 & 1 \\ 1 & 0 & 1 \\ -1 & -1 & -1 \\ 0 & -1 & 0 \end{pmatrix}_{ij} \begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{pmatrix}_{ij} = \frac{1}{6} [0 + 0 + (-1 + 1) + 0] = 0.$$

$$\langle \varphi_1, \varphi_4 \rangle = \frac{1}{3} \frac{1}{3} \sum_{i,j} \begin{pmatrix} 1 & 1 & 1 \\ 1 & 0 & 1 \\ -1 & -1 & -1 \\ 0 & -1 & 0 \end{pmatrix}_{ij} \begin{pmatrix} 0 & 1 & 0 \\ 1 & 1 & 1 \\ 1 & 0 & 1 \\ 1 & 1 & 1 \end{pmatrix}_{ij} = \frac{1}{9} [(0+1+0) + (1+0+1) + (-1+0-1) + (0-1+0)] = 0.$$

$$\langle \varphi_2, \varphi_3 \rangle = \frac{1}{2} \frac{1}{2} \sum_{i,j} \begin{pmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix}_{ij} \begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{pmatrix}_{ij} = \frac{1}{4} \cdot 0 = 0.$$

$$\langle \varphi_2, \varphi_4 \rangle = \frac{1}{2} \frac{1}{3} \sum_{i,j} \begin{pmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix}_{ij} \begin{pmatrix} 0 & 1 & 0 \\ 1 & 1 & 1 \\ 1 & 0 & 1 \\ 1 & 1 & 1 \end{pmatrix}_{ij} = \frac{1}{6} [0 + (1 - 1) + 0 + 0] = 0.$$

$$\langle \varphi_3, \varphi_4 \rangle = \frac{1}{2} \frac{1}{3} \sum_{i,j} \begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{pmatrix}_{ij} \begin{pmatrix} 0 & 1 & 0 \\ 1 & 1 & 1 \\ 1 & 0 & 1 \\ 1 & 1 & 1 \end{pmatrix}_{ij} = \frac{1}{6} [0 + 0 + (1 - 1) + (1 - 1)] = 0.$$

Hence, $\langle \varphi_i, \varphi_j \rangle = \delta_{ij}$ for all $i \neq j$, and together with the unit norms, the set is orthonormal.

(2) Determination of the optimal coefficients In an orthonormal basis, the best approximation of f in $\text{span}\{\varphi_1, \varphi_2, \varphi_3, \varphi_4\}$ (minimizing $\|f - f_a\|^2$) is the orthogonal projection

$$f_a = \sum_{k=1}^4 x_k \varphi_k, \quad x_k = \langle f, \varphi_k \rangle.$$

(3) Calculation of x_1, x_2, x_3, x_4 Taking inner products directly gives

$$x_1 = \langle f, \varphi_1 \rangle = 12, \quad x_2 = \langle f, \varphi_2 \rangle = \frac{27}{2} = 13.5, \quad x_3 = \langle f, \varphi_3 \rangle = 7, \quad x_4 = \langle f, \varphi_4 \rangle = 4.$$

(4) Expression of the approximation f_a

$$f_a = x_1 \varphi_1 + x_2 \varphi_2 + x_3 \varphi_3 + x_4 \varphi_4 = \begin{pmatrix} \frac{43}{4} & \frac{16}{3} & -\frac{11}{4} \\ \frac{145}{12} & \frac{4}{3} & -\frac{17}{12} \\ \frac{5}{6} & -4 & -\frac{37}{6} \\ \frac{29}{6} & -\frac{8}{3} & -\frac{13}{6} \end{pmatrix} \approx \begin{pmatrix} 10.75 & 5.3333 & -2.75 \\ 12.0833 & 1.3333 & -1.4167 \\ 0.8333 & -4.0000 & -6.1667 \\ 4.8333 & -2.6667 & -2.1667 \end{pmatrix}.$$

(5) Approximation quality and error The residual is

$$f - f_a = \begin{pmatrix} \frac{1}{4} & -\frac{1}{3} & -\frac{1}{4} \\ -\frac{1}{12} & -\frac{1}{3} & \frac{5}{12} \\ \frac{1}{6} & 0 & \frac{1}{6} \\ \frac{1}{6} & -\frac{1}{3} & \frac{1}{6} \end{pmatrix}, \quad \|f - f_a\|^2 = \frac{3}{4}, \quad \|f - f_a\| = \frac{\sqrt{3}}{2} \approx 0.8660.$$

Moreover,

$$\|f\| = \sqrt{392} \approx 19.799, \quad \frac{\|f - f_a\|}{\|f\|} \approx 0.04374 \quad (\approx 4.37\%).$$

Thus, f_a is **very close** to f , with a relative error of about 4.4%. Compared with Task 6, here we use a PCA-derived orthonormal basis that captures the main variations of the data and project onto a 4-dimensional subspace. This generally yields a **smaller** approximation error than projecting onto an arbitrarily chosen lower-dimensional subspace (such as the 2-dimensional one in Task 6).

8 Image bases

(1) Projection Function

Listing 2: Projection Function.py

```

1 import numpy as np
2 import scipy.io as sio
3 import matplotlib.pyplot as plt
4 from pathlib import Path
5 from typing import Tuple, List
6
7 # -----
8 # Utility functions: handling MATLAB cell arrays
9 # -----
10 def _from_mat_cell(obj, idx):
11     """
12     Extract an element from a MATLAB cell array loaded by scipy.io.loadmat.
13     """
14     arr = obj
15     if isinstance(arr, np.ndarray) and arr.dtype == object:
16         return arr.flat[idx]
17     return arr
18
19 def _to_np_stack(cell_item) -> np.ndarray:
20     """
21     Convert a cell element to a numpy array with shape (H, W, N).
22     Cast to float32 for numerical stability.
23     """
24     A = np.array(cell_item)
25     if A.ndim != 3:
26         raise ValueError(f"Expected 3-D image stack, got shape {A.shape}")
27     return A.astype(np.float32)
28
29 def _to_np_basis(cell_item) -> np.ndarray:
30     """
31     Convert a basis (19x19x4) to a numpy array with shape (H, W, K).
32     """
33     B = np.array(cell_item)

```

```

34     if B.ndim != 3:
35         raise ValueError(f"Expected 3-D basis, got shape {B.shape}")
36     return B.astype(np.float32)
37
38 # -----
39 # Linear algebra core
40 # -----
41 def flatten_stack(stack_hwN: np.ndarray) -> np.ndarray:
42     """
43     Flatten an image stack (H,W,N) into (D,N), where D=H*W, row-major order
44     .
45     """
46     H, W, N = stack_hwN.shape
47     return stack_hwN.reshape(H*W, N, order="C")
48
49 def flatten_basis(basis_hwk: np.ndarray) -> np.ndarray:
50     """
51     Flatten basis images (H,W,K) into (D,K).
52     """
53     H, W, K = basis_hwk.shape
54     return basis_hwk.reshape(H*W, K, order="C")
55
56 def orthonormalize_columns(E: np.ndarray, eps: float = 1e-8) -> np.ndarray:
57     """
58     Orthonormalize the columns of E (D,K) via QR decomposition, ensuring
59     the basis
60     is numerically orthogonal. Returns Q such that  $Q^T Q = I$ .
61     """
62     Q, R = np.linalg.qr(E)
63     # Fix column signs for consistency (optional)
64     diag_sign = np.sign(np.diag(R))
65     diag_sign[diag_sign == 0] = 1.0
66     Q = Q @ np.diag(diag_sign)
67     return Q
68
69 def project_onto_basis(u: np.ndarray, E: np.ndarray, assume_orthonormal:
70     bool = True) -> Tuple[np.ndarray, float, np.ndarray]:
71     """
72     Project a single vector u (D,) onto the column space of E (D,K).
73     Returns:
74     up   : projection (D,)
75     err  : residual norm ||u - up||_2
76     x    : coefficient vector (K,)
77     If assume_orthonormal=False, solve by least squares.
78     """

```

```

76     if assume_orthonormal:
77         # Orthonormal basis:  $x = E^T u$  ;  $up = E x$ 
78         x = E.T @ u
79         up = E @ x
80     else:
81         # Non-orthogonal basis: solve  $\min_x ||E x - u||_2$ 
82         x, *_ = np.linalg.lstsq(E, u, rcond=None)
83         up = E @ x
84     err = np.linalg.norm(u - up)
85     return up, err, x
86
87 def batch_mean_error(stack_hwN: np.ndarray, basis_hwk: np.ndarray,
88     ensure_orthonormal: bool = True) -> Tuple[float, List[float]]:
89     """
90     Compute the mean error of projecting an image stack onto a given basis.
91     Returns: mean error value and the list of per-image errors.
92     """
93     U = flatten_stack(stack_hwN)          # (D,N)
94     E = flatten_basis(basis_hwk)         # (D,K)
95     if ensure_orthonormal:
96         E = orthonormalize_columns(E)     # safeguard against numerical non-
97                                         # orthogonality
98
99     errs = []
100     for n in range(U.shape[1]):
101         _, err, _ = project_onto_basis(U[:, n], E, assume_orthonormal=True)
102         errs.append(float(err))
103     return float(np.mean(errs)), errs
104
105 # -----
106 # Plotting helpers
107 # -----
108
109 def show_examples(stack_hwN: np.ndarray, title: str, n_show: int = 6):
110     H, W, N = stack_hwN.shape
111     n_show = min(n_show, N)
112     cols = 3
113     rows = int(np.ceil(n_show / cols))
114     plt.figure(figsize=(3*cols, 3*rows))
115     for i in range(n_show):
116         plt.subplot(rows, cols, i+1)
117         plt.imshow(stack_hwN[:, :, i], cmap="gray")
118         plt.axis("off")
119         plt.title(f"#{i+1}")
120     plt.suptitle(title)
121     plt.tight_layout(rect=[0, 0, 1, 0.95])

```

```

119
120 def show_basis(basis_hwk: np.ndarray, title: str):
121     H, W, K = basis_hwk.shape
122     plt.figure(figsize=(3*K, 3))
123     for k in range(K):
124         plt.subplot(1, K, k+1)
125         plt.imshow(basis_hwk[:, :, k], cmap="gray")
126         plt.axis("off")
127         plt.title(f"e{k+1}")
128     plt.suptitle(title)
129     plt.tight_layout(rect=[0, 0, 1, 0.9])
130
131 # -----
132 # Main experiment
133 # -----
134 def main(mat_path: str = "assignment1bases.mat", n_examples_plot: int = 6):
135     mat_path = Path(mat_path)
136     if not mat_path.exists():
137         raise FileNotFoundError(f"File not found {mat_path.resolve()}")
138
139     data = sio.loadmat(mat_path, squeeze_me=False, struct_as_record=False)
140
141     # Extract cell data
142     bases_cell = data["bases"]
143     stacks_cell = data["stacks"]
144
145     # Three bases: bases{1}, bases{2}, bases{3}
146     B1 = _to_np_basis(_from_mat_cell(bases_cell, 0)) # (19,19,4)
147     B2 = _to_np_basis(_from_mat_cell(bases_cell, 1))
148     B3 = _to_np_basis(_from_mat_cell(bases_cell, 2))
149     bases_list = [B1, B2, B3]
150
151     # Two test sets: stacks{1} (general images), stacks{2} (face images)
152     S1 = _to_np_stack(_from_mat_cell(stacks_cell, 0)) # (19,19,400)
153     S2 = _to_np_stack(_from_mat_cell(stacks_cell, 1)) # (19,19,400)
154     stacks_list = [("General (stack 1)", S1), ("Faces (stack 2)", S2)]
155
156     # --- Visualization ---
157     show_examples(S1, "Test Set 1: General", n_examples_plot)
158     show_examples(S2, "Test Set 2: Faces", n_examples_plot)
159     show_basis(B1, "Basis #1 (four elements)")
160     show_basis(B2, "Basis #2 (four elements)")
161     show_basis(B3, "Basis #3 (four elements)")
162
163     # --- Mean error table ---

```

```

164 mean_table = np.zeros((2, 3), dtype=np.float64)
165 for si, (sname, S) in enumerate(stacks_list):
166     for bi, B in enumerate(bases_list):
167         mean_err, _ = batch_mean_error(S, B, ensure_orthonormal=True)
168         mean_table[si, bi] = mean_err
169         print(f"[RESULT] {sname} vs Basis #{bi+1}: mean error = {
            mean_err:.6f}")
170
171     # Format and print table
172     print("\n=== Mean error norms ===")
173     header = "TestSet \\ Basis | #1          #2          #3"
174     print(header)
175     print("-"*len(header))
176     row1 = f"General (1)      | {mean_table[0,0]:.6f} {mean_table[0,1]:.6
        f} {mean_table[0,2]:.6f}"
177     row2 = f"Faces (2)       | {mean_table[1,0]:.6f} {mean_table[1,1]:.6
        f} {mean_table[1,2]:.6f}"
178     print(row1)
179     print(row2)
180
181     plt.show()
182
183 if __name__ == "__main__":
184     main()

```


(2) Test Sets

Some images from Test Set 1: General

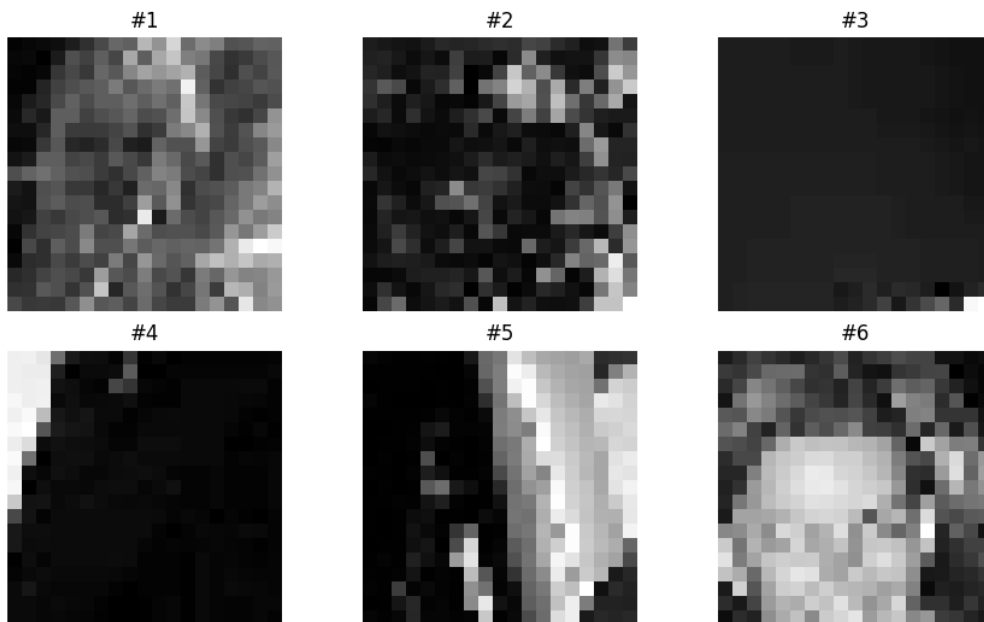


Figure 13: Examples from Test Set 1 (General images).

The first test set (General images) contains a wide variety of 19×19 grayscale images. They show random patterns, textures, blurred shapes, or simple edge-like structures. There is no consistent structure shared among all images, which makes them a general-purpose dataset.

Some images from Test Set 2: Faces

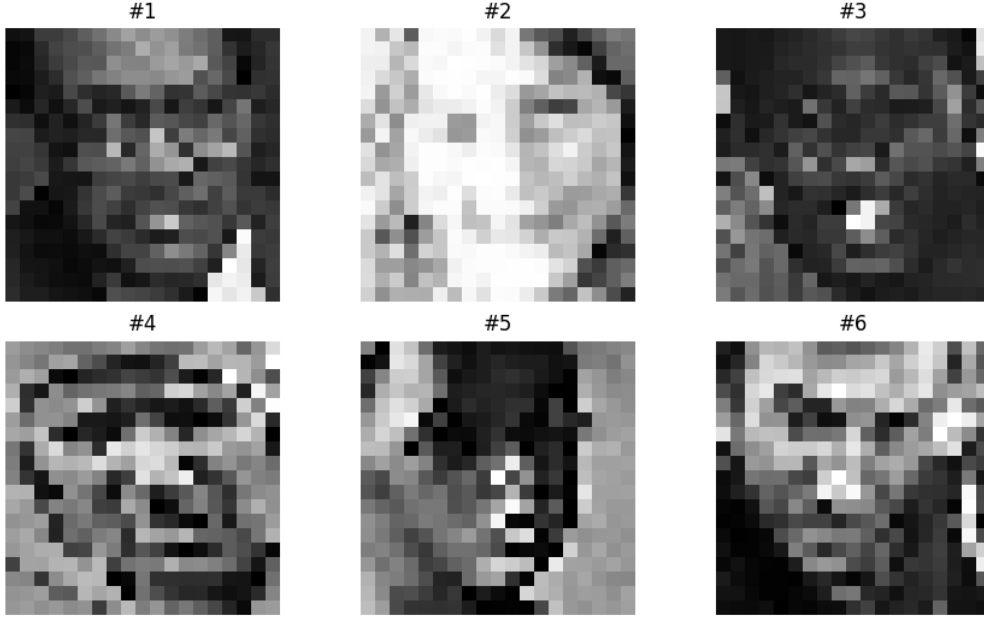


Figure 14: Examples from Test Set 2 (Faces images).

The second test set (Faces) contains low-resolution human faces. Despite the small size, one can clearly observe the rough positions of eyes, nose, and mouth. The images also exhibit variations in brightness and illumination, but they all share a common facial layout.

(3) Basis Sets

Show the four basis images of each basis set:

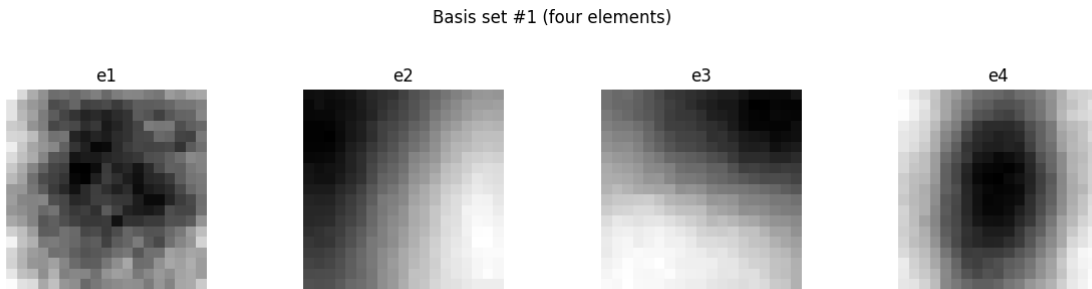


Figure 15: Basis set #1 (four elements).

Basis 1: The four basis elements mainly capture general intensity variations and edge-like patterns, such as horizontal and vertical gradients or blurred blobs. These features are not specific to any class of images but are common in many, which makes this basis perform best on the General test set.

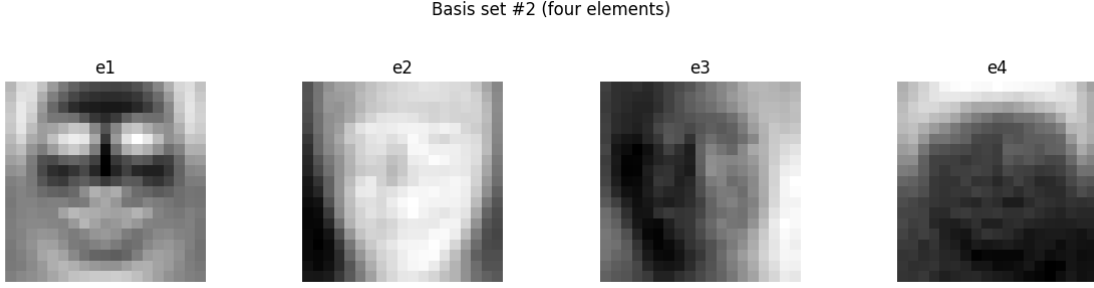


Figure 16: Basis set #2 (four elements).

Basis 2: The four basis elements clearly resemble human face structures, highlighting regions such as eyes and mouth, and representing facial contours under different lighting conditions. They effectively represent the major variations in faces, so this basis performs best on the Faces test set. However, since it is specialized for faces, its performance on non-face images is weaker.

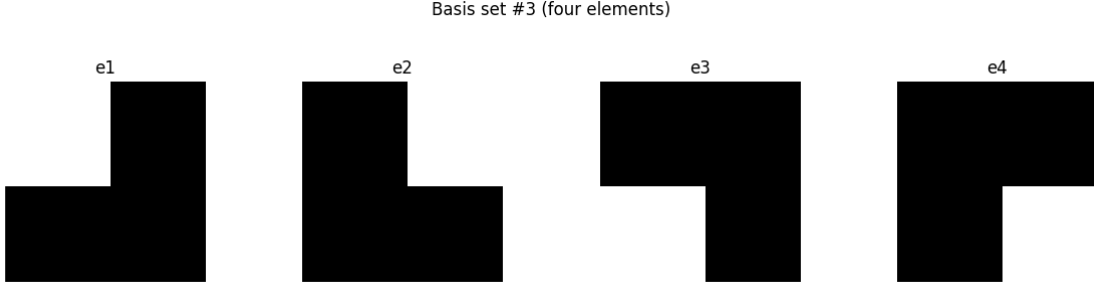


Figure 17: Basis set #3 (four elements).

Basis 3: The four basis elements light up the four corners of the image (top-left, top-right, bottom-left, bottom-right), while the rest remains black. This effectively represents only the coarse brightness distribution across four regions and ignores edges, textures, and semantic structures. Although it has a simple regularity, it is too limited to describe detailed features, so its performance is very poor on both test sets.

(4) Mean Error Results

Present the mean error norms for all six combinations (2 test sets $\times$ 3 bases):

Table 1: Mean reconstruction error norms.				
Test set \ Basis	#1	#2	#3	
General (1)	649.20	795.19	697.32	
Faces (2)	860.48	821.03	944.90	

(5) Discussion

Best Basis for Test Set 1

On the general test set, Basis 1 achieves the lowest mean error. This is because its basis vectors capture common intensity variations and edge structures, making it better suited for general images. In contrast, Basis 2 is specialized for faces and cannot effectively represent non-face patterns, while Basis 3 only encodes corner brightness and performs very poorly.

Best Basis for Test Set 2

On the face test set, Basis 2 performs best with the lowest mean reconstruction error. This is because its four basis vectors align well with the structural variations of human faces, such as symmetry, eye–mouth regions, and illumination changes. Basis 1 can only approximate faces with generic gradients, which is limited, and Basis 3 lacks meaningful detail features, so its performance is also very poor.